

中添加了一个元素，这个操作使保存在 `end` 中的迭代器失效了。这个迭代器不再指向 `v` 中任何元素，或是 `v` 中尾元素之后的位置。



如果在一个循环中插入/删除 `deque`、`string` 或 `vector` 中的元素，不要缓存 `end` 返回的迭代器。

必须在每次插入操作后重新调用 `end()`，而不能在循环开始前保存它返回的迭代器：

```
// 更安全的方法：在每个循环步添加/删除元素后都重新计算 end
while (begin != v.end()) {
    // 做一些处理
    ++begin; // 向前移动 begin，因为我们想在此元素之后插入元素
    begin = v.insert(begin, 42); // 插入新值
    ++begin; // 向前移动 begin，跳过我们刚刚加入的元素
}
```

9.3.6 节练习

练习 9.31: 第 316 页中删除偶数值元素并复制奇数值元素的程序不能用于 `list` 或 `forward_list`。为什么？修改程序，使之也能用于这些类型。

练习 9.32: 在第 316 页的程序中，向下面语句这样调用 `insert` 是否合法？如果不合法，为什么？

```
iter = vi.insert(iter, *iter++);
```

练习 9.33: 在本节最后一个例子中，如果不将 `insert` 的结果赋予 `begin`，将会发生什么？编写程序，去掉此赋值语句，验证你的答案。

练习 9.34: 假定 `vi` 是一个保存 `int` 的容器，其中有偶数值也有奇数值，分析下面循环的行为，然后编写程序验证你的分析是否正确。

```
iter = vi.begin();
while (iter != vi.end())
    if (*iter % 2)
        iter = vi.insert(iter, *iter);
    ++iter;
```

9.4 vector 对象是如何增长的



为了支持快速随机访问，`vector` 将元素连续存储——每个元素紧挨着前一个元素存储。通常情况下，我们不必关心一个标准库类型是如何实现的，而只需关心它如何使用。然而，对于 `vector` 和 `string`，其部分实现渗透到了接口中。

假定容器中元素是连续存储的，且容器的大小是可变的，考虑向 `vector` 或 `string` 中添加元素会发生什么：如果没有空间容纳新元素，容器不可能简单地将它添加到内存中其他位置——因为元素必须连续存储。容器必须分配新的内存空间来保存已有元素和新元素，将已有元素从旧位置移动到新空间中，然后添加新元素，释放旧存储空间。如果我们每添加一个新元素，`vector` 就执行一次这样的内存分配和释放操作，性能会慢到不可接受。

为了避免这种代价，标准库实现者采用了可以减少容器空间重新分配次数的策略。当

不得不获取新的内存空间时，vector 和 string 的实现通常会分配比新的空间需求更大的内存空间。容器预留这些空间作为备用，可用来保存更多的新元素。这样，就不需要每次添加新元素都重新分配容器的内存空间了。

这种分配策略比每次添加新元素时都重新分配容器内存空间的策略要高效得多。其实性能也表现得足够好——虽然 vector 在每次重新分配内存空间时都要移动所有元素，但使用此策略后，其扩张操作通常比 list 和 deque 还要快。

管理容量的成员函数

如表 9.10 所示，vector 和 string 类型提供了一些成员函数，允许我们与它的实现中内存分配部分互动。capacity 操作告诉我们容器在不扩张内存空间的情况下可以容纳多少个元素。reserve 操作允许我们通知容器它应该准备保存多少个元素。

表 9.10: 容器大小管理操作	
shrink_to_fit 只适用于 vector、string 和 deque。	
capacity 和 reserve 只适用于 vector 和 string。	
c.shrink_to_fit()	请将 capacity() 减少为与 size() 相同大小
c.capacity()	不重新分配内存空间的话，c 可以保存多少元素
c.reserve(n)	分配至少能容纳 n 个元素的内存空间



reserve 并不改变容器中元素的数量，它仅影响 vector 预先分配多大的内存空间。

357

只有当需要的内存空间超过当前容量时，reserve 调用才会改变 vector 的容量。如果需求大小大于当前容量，reserve 至少分配与需求一样大的内存空间（可能更大）。

如果需求大小小于或等于当前容量，reserve 什么也不做。特别是，当需求大小小于当前容量时，容器不会退回内存空间。因此，在调用 reserve 之后，capacity 将会大于或等于传递给 reserve 的参数。

这样，调用 reserve 永远也不会减少容器占用的内存空间。类似的，resize 成员函数（参见 9.3.5 节，第 314 页）只改变容器中元素的数目，而不是容器的容量。我们同样不能使用 resize 来减少容器预留的内存空间。

C++
11

在新标准库中，我们可以调用 shrink_to_fit 来要求 deque、vector 或 string 退回不需要的内存空间。此函数指出我们不再需要任何多余的内存空间。但是，具体的实现可以选择忽略此请求。也就是说，调用 shrink_to_fit 也并不保证一定退回内存空间。

capacity 和 size

理解 capacity 和 size 的区别非常重要。容器的 size 是指它已经保存的元素的数目；而 capacity 则是在不分配新的内存空间的前提下它最多可以保存多少元素。

下面的代码展示了 size 和 capacity 之间的相互作用：

```
vector<int> ivec;
// size 应该为 0; capacity 的值依赖于具体实现
cout << " ivec: size: " << ivec.size()
      << " capacity: " << ivec.capacity() << endl;
// 向 ivec 添加 24 个元素
```

```
for (vector<int>::size_type ix = 0; ix != 24; ++ix)
    ivec.push_back(ix);

// size 应该为 24; capacity 应该大于等于 24, 具体值依赖于标准库实现
cout << "ivec: size: " << ivec.size()
    << " capacity: " << ivec.capacity() << endl
```

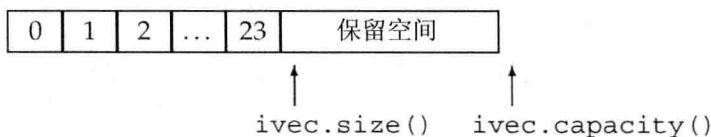
当在我们的系统上运行时, 这段程序得到如下输出:

```
ivec: size: 0 capacity: 0
ivec: size: 24 capacity: 32
```

我们知道一个空 vector 的 size 为 0, 显然在我们的标准库实现中一个空 vector 的 capacity 也为 0。当向 vector 中添加元素时, 我们知道 size 与添加的元素数目相等。而 capacity 至少与 size 一样大, 具体会分配多少额外空间则视标准库具体实现而定。在我们的标准库实现中, 每次添加 1 个元素, 共添加 24 个元素, 会使 capacity 变为 32。

◀ 358

可以想象 ivec 的当前状态如下图所示:



现在可以预分配一些额外空间:

```
ivec.reserve(50); // 将 capacity 至少设定为 50, 可能会更大
// size 应该为 24; capacity 应该大于等于 50, 具体值依赖于标准库实现
cout << "ivec: size: " << ivec.size()
    << " capacity: " << ivec.capacity() << endl;
```

程序的输出表明 reserve 严格按照我们需求的大小分配了新的空间:

```
ivec: size: 24 capacity: 50
```

接下来可以用光这些预留空间:

```
// 添加元素用光多余容量
while (ivec.size() != ivec.capacity())
    ivec.push_back(0);
// capacity 应该未改变, size 和 capacity 不相等
cout << "ivec: size: " << ivec.size()
    << " capacity: " << ivec.capacity() << endl;
```

程序输出表明此时我们确实用光了预留空间, size 和 capacity 相等:

```
ivec: size: 50 capacity: 50
```

由于我们只使用了预留空间, 因此没有必要为 vector 分配新的空间。实际上, 只要没有操作需求超出 vector 的容量, vector 就不能重新分配内存空间。

如果我们现在再添加一个新元素, vector 就不得不重新分配空间:

```
ivec.push_back(42); // 再添加一个元素
// size 应该为 51; capacity 应该大于等于 51, 具体值依赖于标准库实现
cout << "ivec: size: " << ivec.size()
    << " capacity: " << ivec.capacity() << endl;
```

这段程序的输出为

359

`ivec: size: 51 capacity: 100`

这表明 `vector` 的实现采用的策略似乎是在每次需要分配新内存空间时将当前容量翻倍。

可以调用 `shrink_to_fit` 来要求 `vector` 将超出当前大小的多余内存退回给系统:

```
ivec.shrink_to_fit(); // 要求归还内存
// size 应该未改变; capacity 的值依赖于具体实现
cout << "ivec: size: " << ivec.size()
    << " capacity: " << ivec.capacity() << endl;
```

调用 `shrink_to_fit` 只是一个请求, 标准库并不保证退还内存。



每个 `vector` 实现都可以选择自己的内存分配策略。但是必须遵守的一条原则是: 只有当迫不得已时才可以分配新的内存空间。

只有在执行 `insert` 操作时 `size` 与 `capacity` 相等, 或者调用 `resize` 或 `reserve` 时给定的大小超过当前 `capacity`, `vector` 才可能重新分配内存空间。会分配多少超过给定容量的额外空间, 取决于具体实现。

虽然不同的实现可以采用不同的分配策略, 但所有实现都应遵循一个原则: 确保用 `push_back` 向 `vector` 添加元素的操作有高效率。从技术角度说, 就是通过在一个初始为空的 `vector` 上调用 n 次 `push_back` 来创建一个 n 个元素的 `vector`, 所花费的时间不能超过 n 的常数倍。

9.4 节练习

练习 9.35: 解释一个 `vector` 的 `capacity` 和 `size` 有何区别。

练习 9.36: 一个容器的 `capacity` 可能小于它的 `size` 吗?

练习 9.37: 为什么 `list` 或 `array` 没有 `capacity` 成员函数?

练习 9.38: 编写程序, 探究在你的标准库实现中, `vector` 是如何增长的。

练习 9.39: 解释下面程序片段做了什么:

```
vector<string> svec;
svec.reserve(1024);
string word;
while (cin >> word)
    svec.push_back(word);
svec.resize(svec.size()+svec.size()/2);
```

练习 9.40: 如果上一题中的程序读入了 256 个词, 在 `resize` 之后容器的 `capacity` 可能是多少? 如果读入了 512 个、1000 个或 1048 个词呢?

360

9.5 额外的 `string` 操作

除了顺序容器共同的操作之外, `string` 类型还提供了一些额外的操作。这些操作中的大部分要么是提供 `string` 类和 C 风格字符数组之间的相互转换, 要么是增加了允许我们用下标代替迭代器的版本。